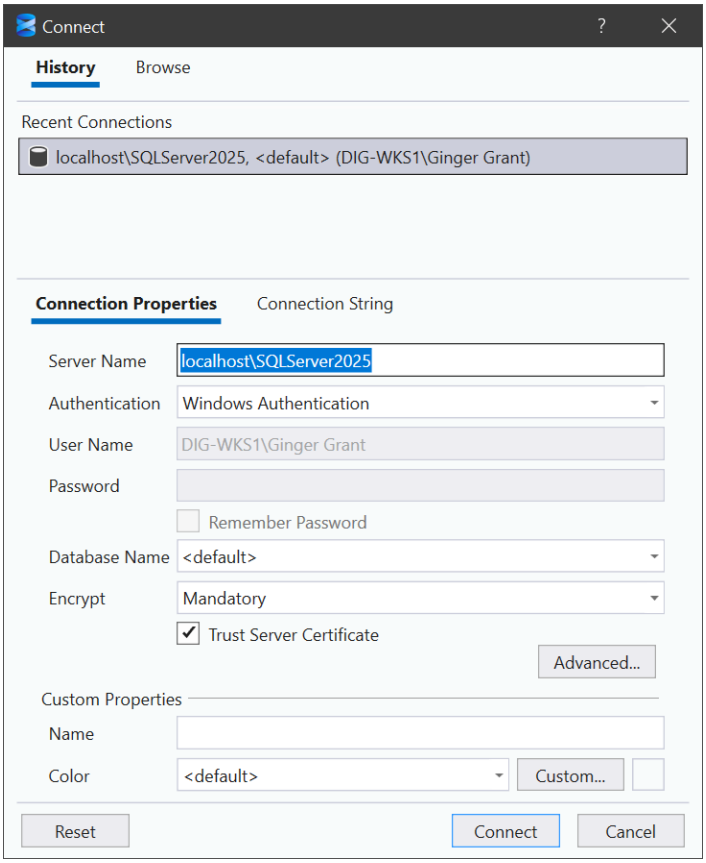
Exercise 1 - Introduction to SQL Server

1. Open up SQL Server Management Studio and login to the SQL Server 2025 you have installed on your desktop



2. Select the database AdventureworksDW2020, which was included in the gitrepo so that you could load it to your local PC, then click on New Query

3. Validate that you are using SQL Server 2025 with this query

```
SELECT @@VERSION;
```

4. Check to see what version the database is

```
SELECT compatibility_level
FROM sys.databases
WHERE name = 'AdventureWorksDW2020'
```

5. If the column is not 170, and if you downloaded the version from github, it isn't, you will need to change the compatibility. Run the following command then check the results again, and you will see a compatibility_level of 170

```
ALTER DATABASE AdventureWorksDW2020
SET COMPATIBILITY_LEVEL = 170;

SELECT compatibility_level
FROM sys.databases
WHERE name = 'AdventureWorksDW2020'
```

6. To ensure that the emails stored in the database are valid, run the next query to validate the emails using REGEXP_LIKE, a new T-SQL command

```
SELECT
    CustomerKey,
```

```
    EmailAddress,
    CASE
        WHEN REGEXP_LIKE(EmailAddress, '^[A-Za-z0-9._%+\-]+@[A-Za-z0-9.\-
]+\.[A-Za-z]{2,}$')
        THEN 'Valid'
        ELSE 'Invalid'
    END AS EmailValidation
FROM dbo.DimCustomer
WHERE EmailAddress IS NOT NULL
```

7. Review the results. To show only the invalid records, add this line to the end of the SQL statement

```
AND NOT REGEXP_LIKE(EmailAddress, '^[A-Za-z0-9._%+\-]+@[A-Za-z0-9.\-]+\.[A-
Za-z]{2,}$')
```

In the results, you will see the invalid email addresses.

8. Next let's examine the new JSON features in SQL Server 2025. First we are going to create a new table in the database using the new JSON data type.

```
CREATE TABLE CustomerProfiles (
    ProfileID INT PRIMARY KEY IDENTITY,
    CustomerKey INT NOT NULL,
    ProfileData JSON NOT NULL,  -- Native JSON type (NEW)
    CreatedDate DATETIME2 DEFAULT GETDATE()
);
GO
```

9. Next let's insert some data into the table

```
INSERT INTO dbo.CustomerProfiles (CustomerKey, ProfileData)
SELECT TOP (20)
    CustomerKey,
    JSON_OBJECT(
        'customerId' : CustomerKey,
        'name'       : CONCAT(FirstName, ' ', LastName),
        'email'      : EmailAddress,
        'phone'      : Phone
    ) AS ProfileData
FROM dbo.DimCustomer
WHERE FirstName IS NOT NULL;
```

10. Now let's query the JSON data

```
SELECT JSON_VALUE(ProfileData, '$.name')
FROM CustomerProfiles
WHERE JSON_VALUE(ProfileData, '$.email') LIKE '%adventure%';
```

Exercise 2 - SQL Database AI in SSMS 22

Explore, document, and build with GitHub Copilot on SQL Server 2025. A self-guided, step-by-step tutorial.

About this lab

Series: Follows Exercise 1, SQL Server 2025 setup and what's new.

Based on: SSMS 22 Unlocked, Features You've Been Waiting For.

You will use: SSMS 22, local SQL Server 2025 default instance, GitHub Copilot.

Format: Self-paced. Read, run, and verify at your own pace.

Estimated time: 60 minutes (setup 10 minutes, explore 15 minutes, build with AI 35 minutes).

You will build: A sample database, table documentation, one view, one stored procedure.

0. Before you begin

In Exercise 1, you stood up SQL Server 2025 and toured what's new in the engine. In this lab you connect to that local instance with SQL Server Management Studio (SSMS) 22, explore a schema, and then put GitHub Copilot to work on three everyday developer tasks: documenting tables, designing a view, and writing a stored procedure. The lesson underneath every step is the same: AI accelerates SQL development, but you still own correctness.

What you'll accomplish

- Connect to a local SQL Server 2025 default instance from SSMS 22.
- Create a small sample database and explore its user tables and metadata.
- Use GitHub Copilot (Ask mode) to document tables from real schema metadata.
- Generate, review, and test a reporting view (dbo.vw_OrderSummary).
- Generate, review, and test a parameterized stored procedure (dbo.usp_GetOrderSummaryByDateRange).
- Compare AI-assisted authoring with manual authoring for speed and quality.

Prerequisites

- SSMS 22 installed (22.7 or later recommended, since the SQL Formatter and current Copilot features arrived in 22.7).
- A local SQL Server 2025 instance running as the default instance (from Exercise 1).
- GitHub Copilot available in your coding environment. Copilot is built into SSMS 22. If it isn't enabled for you, run the same prompts in a supported editor side by side and paste the results into SSMS.

Don't have a sample database yet? Section 1 below gives you a single create script that builds everything this lab needs. If Exercise 1 already left you a database, adapt the queries to your own table names.

How to read this guide

Each section is a short, runnable step. Watch for these labeled notes:

Tip: a shortcut, better prompt, or good habit worth adopting.

Checkpoint: a quick way to confirm the step worked before moving on. If your result doesn't match, fix it here.

Watch out: a common mistake or something Copilot can get wrong. Review before you trust it.

Code you should run appears in monospace listings, numbered as Listing N. Screenshots show what the SSMS 22 interface should look like at each step (your object names and paths will differ slightly).

1. Set up the lab database

This lab uses a small sales and order-entry schema of five user tables. Run the create script below once against your local instance to build the database and load a little sample data, so every query, the view, and the stored procedure return meaningful results.

Run the create script

1. In SSMS, open a new query window: select New Query on the toolbar (or press Ctrl+N).
2. Paste Listing 1 into the editor.
3. Select Execute (or press F5). The Messages pane should read: Commands completed successfully.

```
/*
=====================================================================
Session 2 Lab - sample database setup
Creates SalesLabDB with five user tables and a little sample data.
Run ONCE against your local SQL Server 2025 default instance.
=====================================================================
*/
IF DB_ID('SalesLabDB') IS NULL
CREATE DATABASE SalesLabDB;
GO
USE SalesLabDB;
GO

CREATE TABLE dbo.Customers (
    CustomerID int IDENTITY(1,1) PRIMARY KEY,
    CustomerName nvarchar(200) NOT NULL,
    Email nvarchar(256) NULL,
    City nvarchar(100) NULL,
    CreatedDate datetime2(0) NOT NULL DEFAULT SYSUTCDATETIME()
);

CREATE TABLE dbo.Products (
    ProductID int IDENTITY(1,1) PRIMARY KEY,
    ProductName nvarchar(200) NOT NULL,
    Category nvarchar(100) NULL,
    UnitPrice money NOT NULL
);

CREATE TABLE dbo.Employees (
    EmployeeID int IDENTITY(1,1) PRIMARY KEY,
    FullName nvarchar(200) NOT NULL,
    Title nvarchar(100) NULL,
    HireDate date NULL
);

CREATE TABLE dbo.Orders (
    OrderID int IDENTITY(1001,1) PRIMARY KEY,
    CustomerID int NOT NULL
        CONSTRAINT FK_Orders_Customers REFERENCES dbo.Customers(CustomerID),
    OrderDate date NOT NULL,
    Status nvarchar(30) NOT NULL DEFAULT 'Processing'
);

CREATE TABLE dbo.OrderItems (
    OrderItemID int IDENTITY(1,1) PRIMARY KEY,
```

```
    OrderID int NOT NULL
        CONSTRAINT FK_OrderItems_Orders REFERENCES dbo.Orders(OrderID),
    ProductID int NOT NULL
        CONSTRAINT FK_OrderItems_Products REFERENCES dbo.Products(ProductID),
    Quantity int NOT NULL,
    UnitPrice money NOT NULL
);
GO

/* ---- sample data ---- */
INSERT dbo.Customers (CustomerName, Email, City, CreatedDate) VALUES
('Contoso Ltd', 'orders@contoso.com', 'Seattle', '2025-11-03'),
('Fabrikam Inc', 'buyer@fabrikam.com', 'Portland', '2025-12-15'),
('Adventure Works', 'purchasing@adventure-works.com', 'Denver', '2026-01-
20'),
('Northwind Traders', 'ap@northwind.com', 'Austin', '2026-02-08'),
('Tailspin Toys', 'orders@tailspintoys.com', 'Chicago', '2026-03-12');

INSERT dbo.Products (ProductName, Category, UnitPrice) VALUES
('Wireless Mouse', 'Accessories', 24.99),
('Mechanical Keyboard', 'Accessories', 89.99),
('27" Monitor', 'Displays', 329.00),
('USB-C Dock', 'Accessories', 189.50),
('Laptop Stand', 'Accessories', 39.95),
('Webcam 1080p', 'Peripherals', 59.00);

INSERT dbo.Employees (FullName, Title, HireDate) VALUES
('Dana Reyes', 'Sales Manager', '2024-05-13'),
('Sam Carter', 'Account Executive', '2025-02-01'),
('Priya Nair', 'Support Specialist', '2025-09-22');

INSERT dbo.Orders (CustomerID, OrderDate, Status) VALUES
(1,'2026-02-02','Shipped'), (2,'2026-02-14','Shipped'),
(1,'2026-03-05','Processing'), (3,'2026-03-19','Shipped'),
(4,'2026-04-01','Shipped'), (5,'2026-04-22','Processing'),
(2,'2026-05-10','Shipped');

INSERT dbo.OrderItems (OrderID, ProductID, Quantity, UnitPrice) VALUES
(1001,1,2,24.99),(1001,3,1,329.00),
(1002,2,1,89.99),(1002,5,3,39.95),
(1003,4,1,189.50),
(1004,3,2,329.00),(1004,6,2,59.00),(1004,1,4,24.99),
(1005,2,2,89.99),
(1006,5,1,39.95),(1006,6,1,59.00),
(1007,4,2,189.50),(1007,1,1,24.99);
GO
```

Listing 1. Create the SalesLabDB sample database, tables, and seed data.

Checkpoint: Run SELECT COUNT(*) FROM dbo.Orders; you should get 7 rows. If the database already existed, the CREATE DATABASE line is skipped and the tables and inserts still run.

Re-running the lab: To reset between attempts, run DROP DATABASE IF EXISTS SalesLabDB; and execute Listing 1 again from a clean slate.

2. Connect to the local default instance

SSMS 22 introduces a Modern Connection Dialog with a searchable connection history and a properties panel. Use it to connect to the SQL Server 2025 instance you set up in Exercise 1.

4. Launch SSMS 22. The Connect dialog opens automatically (or select Connect, then Database Engine, in Object Explorer).
5. For Server name, type a period (.) or localhost. Both point at the local default instance on this machine.
6. Set Authentication to Windows Authentication (or SQL login if that's how Exercise 1 was configured).
7. Leave Encrypt set to Mandatory and select Trust Server Certificate for a local dev box, then choose Connect.

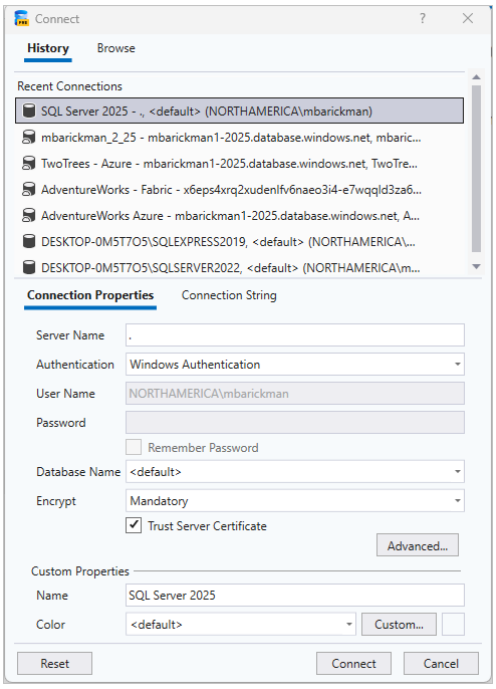


Figure 1. The SSMS 22 Modern Connection Dialog. Server name "." targets the local default instance; Trust Server Certificate is enabled for local development.

Named instance instead? A period (.) or "localhost" reaches the default instance. If Exercise 1 installed a named instance, connect with .\InstanceName (for example .\SQL2025).

Checkpoint: Object Explorer shows your server node at the top, for example localhost (SQL Server 17.0). You're connected.

3. Explore the schema in Object Explorer

Before you ask AI to generate anything, get your bearings. Developers almost always start in three places: Tables, Views, and Programmability, Stored Procedures.

8. Expand Databases, then SalesLabDB, then Tables.
9. Note the five user tables (dbo.Customers, dbo.Products, dbo.Orders, dbo.OrderItems, dbo.Employees). These are separate from the greyed-out System Databases and system objects.
10. Peek at Views and Programmability, then Stored Procedures. You'll add one of each later in this lab.

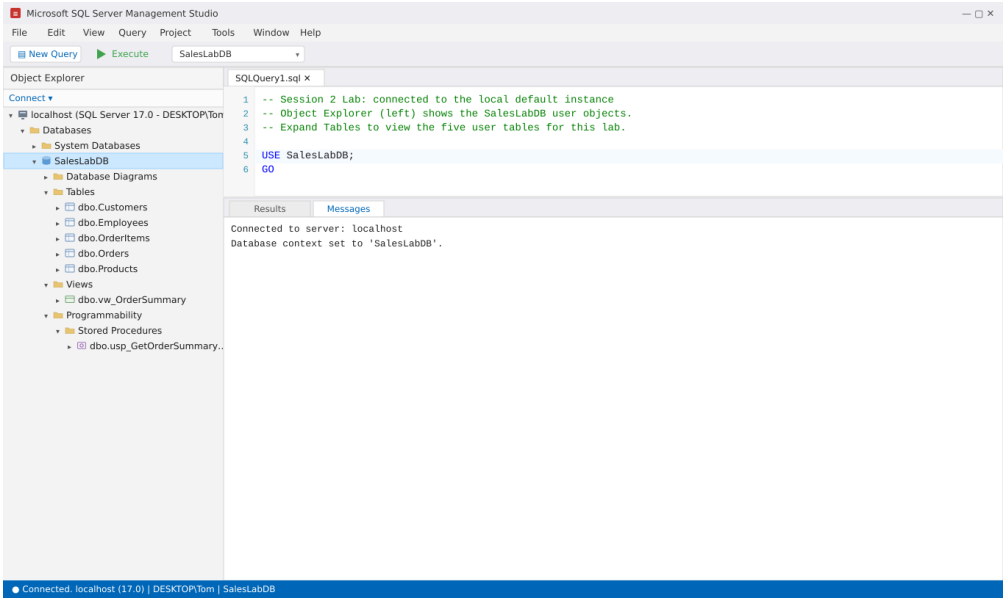


Figure 2. Object Explorer expanded to the five SalesLabDB user tables, with Views and Programmability where your new objects will appear.

Group by schema: SSMS 22 can group Object Explorer nodes by schema, which keeps large databases readable. Right-click the Tables folder to explore the grouping and filter options.

4. Inspect metadata with catalog views

Clicking through Object Explorer is fine for five tables, but catalog views scale and give you exactly the context Copilot needs. Gather that context now.

4.1 List the user tables

```
SELECT
    s.name AS schema_name,
    t.name AS table_name
FROM sys.tables t
JOIN sys.schemas s
    ON t.schema_id = s.schema_id
WHERE t.is_ms_shipped = 0
ORDER BY s.name, t.name;
```

Listing 2. List user tables (excludes Microsoft-shipped system objects).

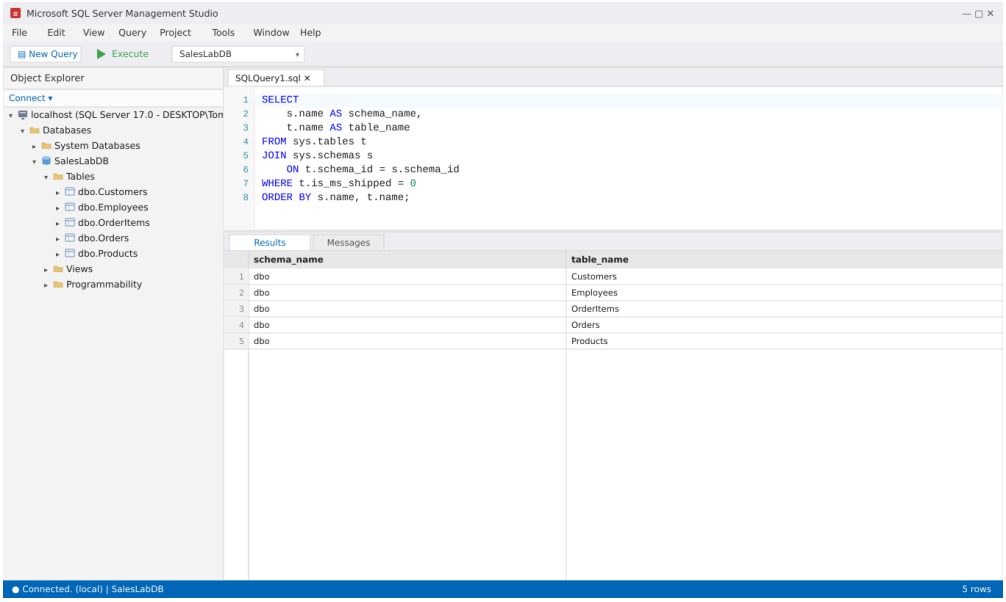


Figure 3. Running Listing 2 returns the five user tables in the Results grid.

4.2 Inspect columns and data types

```
SELECT
    t.name AS table_name,
    c.column_id,
    c.name AS column_name,
    ty.name AS data_type,
    c.max_length,
    c.is_nullable
FROM sys.tables t
JOIN sys.columns c ON t.object_id = c.object_id
JOIN sys.types ty ON c.user_type_id = ty.user_type_id
WHERE t.name = 'Customers'
ORDER BY c.column_id;
```

Listing 3. Column metadata for a single table (change the WHERE clause to inspect others).

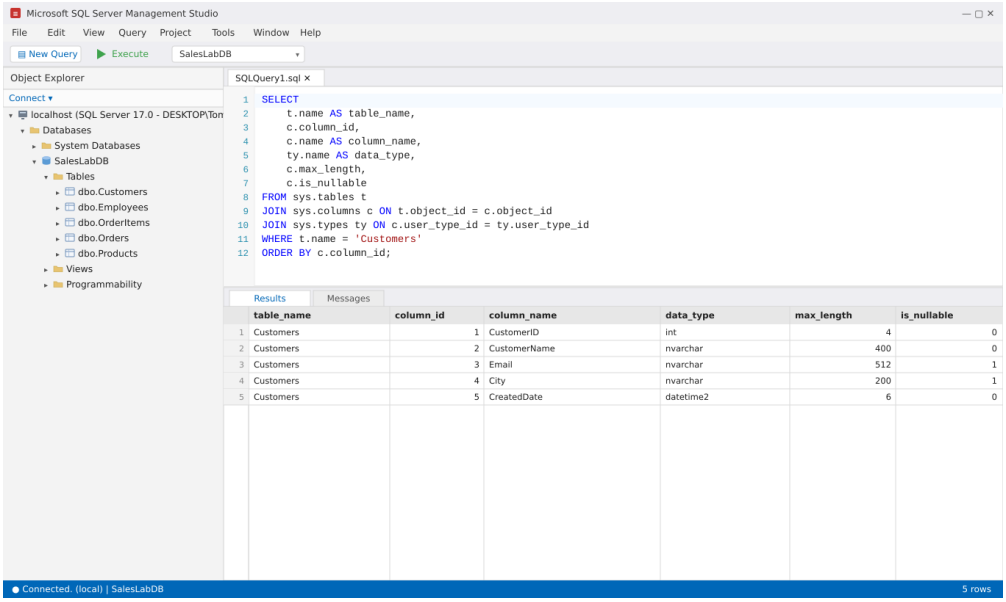


Figure 4. Column metadata for dbo.Customers, including data type, length, and nullability.

Why max_length looks doubled: sys.columns.max_length is reported in bytes. Because nvarchar stores 2 bytes per character, nvarchar(200) shows 400, nvarchar(256) shows 512, and so on. A value of -1 means MAX. datetime2(0) is 6 bytes.

4.3 Find primary and foreign keys

```
SELECT
    OBJECT_SCHEMA_NAME(parent_object_id) AS schema_name,
    OBJECT_NAME(parent_object_id) AS table_name,
    name AS constraint_name,
    type_desc
FROM sys.objects
WHERE type IN ('PK', 'F')  -- PRIMARY KEY and FOREIGN KEY constraints
ORDER BY schema_name, table_name, type_desc;
```

Listing 4. Discover the keys and relationships that make a good reporting view.

Your task, before you move on:

- Identify 3 to 4 tables you understand well (Customers, Orders, OrderItems, Products are ideal).
- Note the likely primary keys and the foreign keys that connect Orders to Customers and OrderItems to Orders and Products.
- Decide which tables you'll combine into a reporting view (you'll use Orders, Customers, and OrderItems in Section 6).

5. Document your tables with GitHub Copilot (Ask mode)

SSMS 22 offers two Copilot modes. Ask mode answers questions and generates snippets; use it when you need knowledge, are exploring, or want a quick answer. Agent mode runs multi-step investigations and needs your approval for every query. Documentation is a perfect Ask-mode task.

5.1 Turn metadata into documentation

11. Open the GitHub Copilot chat panel in SSMS (or a supported editor) and make sure the mode is set to Ask.
12. Run Listing 3 for each table you chose, and copy the results.

13. Paste the metadata into the prompt below and send it.

```
Based on the following SQL Server schema metadata, write concise
Markdown documentation for each user table. For each table include:
- business purpose
- key columns and their meaning
- likely primary key and foreign key relationships
- developer usage notes

[paste your Listing 3 results for each table here]
```

Listing 5. An Ask-mode prompt that grounds Copilot in your real schema.

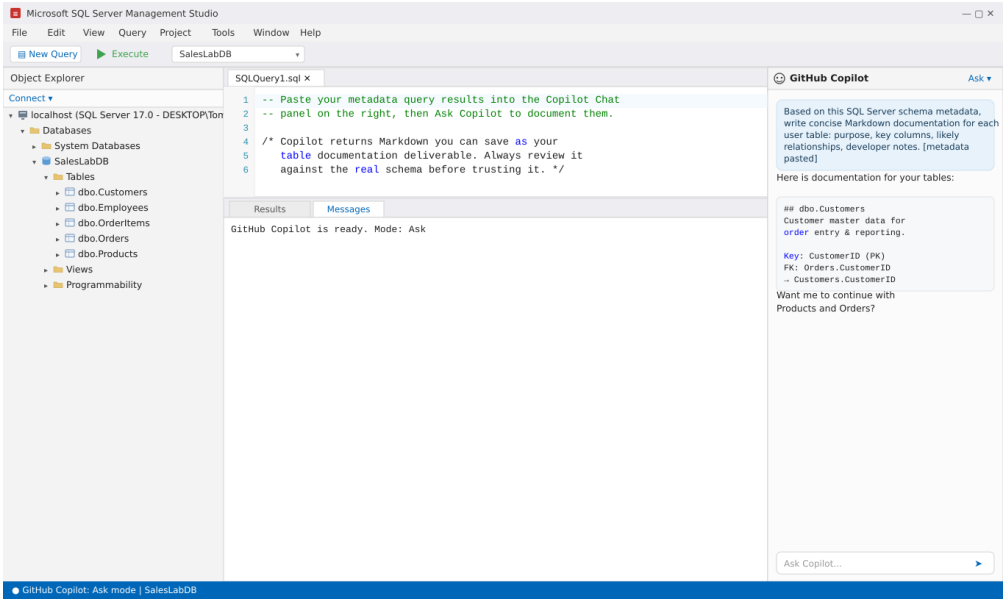


Figure 5. Copilot (Ask mode) returns Markdown documentation grounded in the pasted metadata.

5.2 What good output looks like

Aim for a short, skimmable artifact per table, something like this:

```
## dbo.Customers
Customer master data for order entry and reporting.

### Key columns
- CustomerID : unique customer identifier (primary key)
- CustomerName : customer display name
- Email : contact email
- CreatedDate : record creation timestamp (UTC)

### Relationships
- One customer has many orders (Orders.CustomerID references
Customers.CustomerID)

### Developer notes
Use for customer lookups and as the join target from Orders.
```

Listing 6. Target documentation format for one table.

You own correctness: Copilot improves speed, not certainty. Check every generated claim against the real schema: does the primary key it named actually exist? Are the relationships right? A good prompt (table names, column lists, known relationships, desired format) dramatically improves the first draft, but you still validate it.

Deliverable: Documentation for at least three user tables, saved as Markdown or text.

6. Build a reporting view with Copilot

Views give you consistent, reusable access patterns. You'll create dbo.vw_OrderSummary, one row per order with the customer name, item count, and order total.

6.1 Ask Copilot for the view

```
Generate a SQL Server CREATE VIEW statement for dbo.vw_OrderSummary.
Use dbo.Orders, dbo.Customers, and dbo.OrderItems. Return:
- OrderID
- OrderDate
- CustomerID
- CustomerName
- COUNT of items AS ItemCount
- SUM(Quantity*UnitPrice) AS OrderTotal
Join appropriately and GROUP BY the non-aggregated columns.
```

Listing 7. Copilot prompt for the reporting view.

6.2 Review, then create the view

Read the generated SQL before you run it. Confirm the joins, the aggregation, and the GROUP BY. It should look like this:

```
CREATE OR ALTER VIEW dbo.vw_OrderSummary
AS
SELECT
    o.OrderID,
    o.OrderDate,
    o.CustomerID,
    c.CustomerName,
    COUNT(oi.OrderItemID) AS ItemCount,
    SUM(oi.Quantity * oi.UnitPrice) AS OrderTotal
FROM dbo.Orders o
JOIN dbo.Customers c ON o.CustomerID = c.CustomerID
JOIN dbo.OrderItems oi ON o.OrderID = oi.OrderID
GROUP BY o.OrderID, o.OrderDate, o.CustomerID, c.CustomerName;
GO
```

Listing 8. The view. CREATE OR ALTER lets you re-run it safely while iterating.

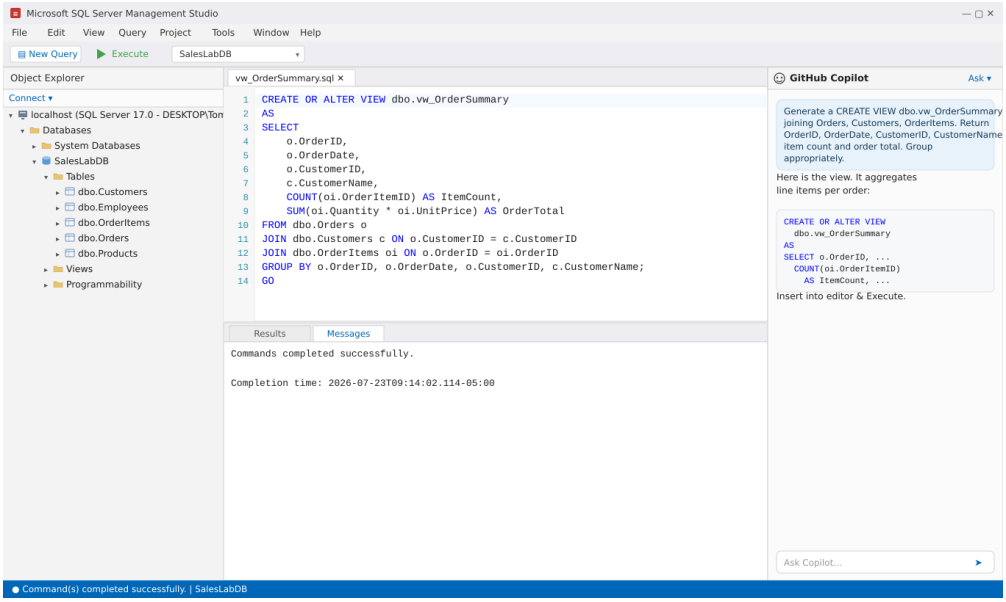


Figure 6. Executing the CREATE VIEW statement; the Messages pane confirms success.

6.3 Test the view

```
SELECT TOP (20) *
FROM dbo.vw_OrderSummary
ORDER BY OrderDate DESC;
```

Listing 9. Query the view to confirm it returns one summarized row per order.

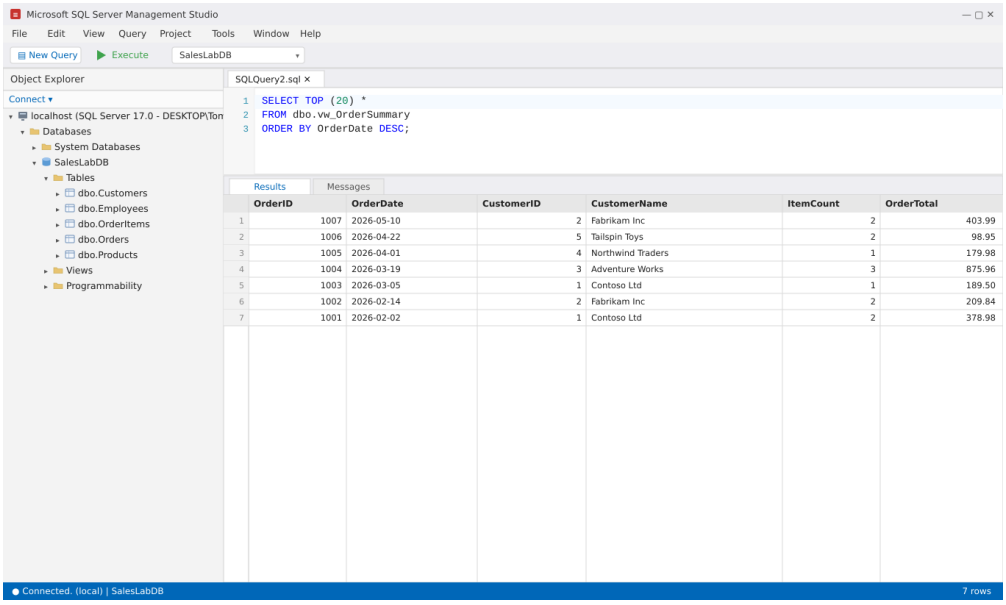


Figure 7. The view returns seven summarized orders, newest first, with per-order item counts and totals.

Checkpoint: Order 1004 (Adventure Works) shows ItemCount 3 and OrderTotal 875.96. If your totals differ, re-check the OrderItems join and the SUM expression.

7. Build a stored procedure with Copilot

Stored procedures package reusable, parameterized logic. You'll create dbo.usp_GetOrderSummaryByDateRange. It returns orders in a date range, with an optional customer filter and basic input validation.

7.1 Ask Copilot for the procedure

```
Generate a SQL Server stored procedure named
dbo.usp_GetOrderSummaryByDateRange.

Parameters:
- @StartDate date
- @EndDate date
- @CustomerID int = NULL

Requirements:
- Validate that @StartDate <= @EndDate (THROW if not)
- Query dbo.vw_OrderSummary
- Filter by OrderDate between the parameters
- Apply the customer filter only when @CustomerID is provided
- Sort by OrderDate descending
```

Listing 10. Copilot prompt for the parameterized stored procedure.

7.2 Review, then create the procedure

Check the validation, the optional-parameter pattern, and the sort order before running:

```
CREATE OR ALTER PROCEDURE dbo.usp_GetOrderSummaryByDateRange
    @StartDate date,
    @EndDate date,
    @CustomerID int = NULL
AS
BEGIN
    SET NOCOUNT ON;

    IF @StartDate > @EndDate
        THROW 50001, 'StartDate must be less than or equal to EndDate.', 1;

    SELECT
        OrderID, OrderDate, CustomerID, CustomerName,
        ItemCount, OrderTotal
    FROM dbo.vw_OrderSummary
    WHERE OrderDate >= @StartDate
      AND OrderDate <= @EndDate
      AND (@CustomerID IS NULL OR CustomerID = @CustomerID)
    ORDER BY OrderDate DESC;
END;
GO
```

Listing 11. The stored procedure, reusing the view from Section 6.

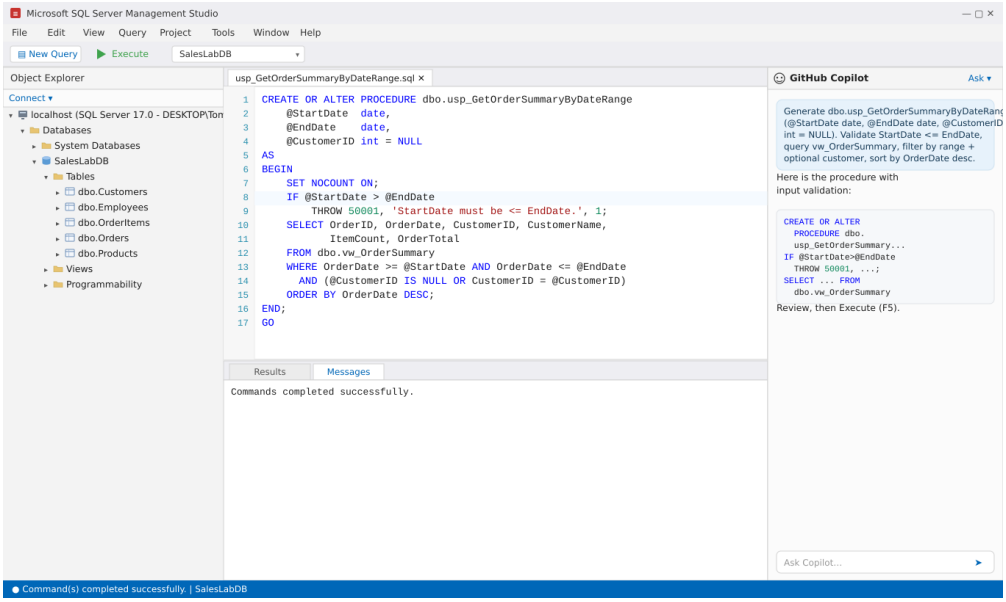


Figure 8. Creating the stored procedure. The IF/THROW guard validates the date range before querying.

7.3 Test the procedure

```
EXEC dbo.usp_GetOrderSummaryByDateRange
     @StartDate = '2026-03-01',
     @EndDate = '2026-04-30',
     @CustomerID = NULL;
```

Listing 12. Execute the procedure for a two-month window across all customers.

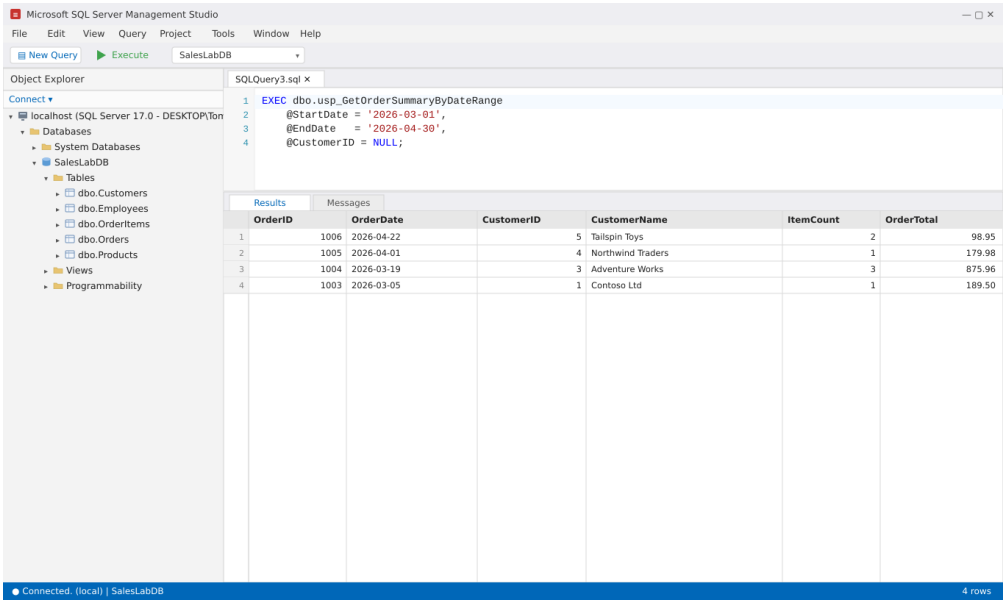


Figure 9. The procedure returns the four orders that fall inside the requested date range, newest first.

Try the validation and the filter: Swap the dates so @StartDate is after @EndDate; you should get error 50001. Then set @CustomerID = 1 to see only Contoso Ltd's orders in range.

8. Success criteria and debrief

You've completed the lab when you can show

- A working connection to your local SQL Server 2025 instance in SSMS 22.
- A metadata query listing the user tables (Listing 2).
- Documentation for at least three user tables (Section 5).
- A new view, dbo.vw_OrderSummary, that executes and returns summarized orders.
- A new stored procedure, dbo.usp_GetOrderSummaryByDateRange, that executes with and without a customer filter.

Debrief, discuss or reflect

14. What did Copilot do well?
15. What still required human review?
16. Was the generated SQL correct on the first pass?
17. Where did adding real schema context improve the result?
18. Would you trust AI to write production SQL without review? Why or why not?

Finished early? Stretch goals

Ask Copilot to improve the stored procedure with one of these, then review the result:

- Default the date range to the last 30 days when parameters are omitted.
- Add an optional @MinOrderTotal filter.
- Add a second sort option (for example, by OrderTotal descending).
- Add inline comments that explain the logic for the next developer.

9. Reference: what's new in SSMS 22

A quick tour of the SSMS 22 features behind this lab, drawn from "SSMS 22 Unlocked."

Modern Connection Dialog: Searchable connection history plus a properties panel (used in Section 2).

GitHub Copilot in SSMS: Code completions, Ask mode, and Agent mode using your existing Copilot subscription, the same account across IDEs.

Ask mode: Answers questions and generates snippets. Best for "What is...", "How do I...", "Why does..."

Agent mode: Runs multi-step investigations (for example, "analyze blocking for the past hour"); READ_ONLY by default and approves every query.

Execution Context: 22.7 adds running Copilot queries as a specific database user or login via EXECUTE AS (needs impersonation permission).

SQL Formatter (Preview): ScriptDOM-based formatting, configurable under Tools, then Options, then SQL Formatter; format on demand or on save.

Group by Schema: Organizes Object Explorer nodes by schema for readability.

Schema Compare: Compare database, SQL project, or .dacpac sources and targets.

Query Hint Recommendation: Suggests query hints to help tune problem queries.

Other touches: Export results, zoom the results grid, open a plan in a new tab, ARM64 support, new themes, faster startup.

Ask mode vs Agent mode, quick guide

Use Ask when you: need knowledge or a quick answer, are exploring a concept, ask "What is...", "How do I...", "Why does...", or want a snippet to adapt yourself.

Use Agent when you: need analysis across multiple steps, would normally run several queries or scripts, ask "Investigate...", "Analyze...", "Assess...", or want Copilot to do the work and propose actions.

Rule of thumb: Ask mode informs, Agent mode acts.

Resources

- SSMS documentation: learn.microsoft.com/ssms
- GitHub Copilot in SSMS, video playlist: aka.ms/ssms-ghcp-playlist
- Send feedback or log bugs: aka.ms/ssms-feedback

Appendix A. All scripts in one place

For convenience, the objects you build in this lab. Run Listing 1 first, then the view, then the procedure.

A.1 Reporting view

```
CREATE OR ALTER VIEW dbo.vw_OrderSummary
AS
SELECT
    o.OrderID, o.OrderDate, o.CustomerID, c.CustomerName,
    COUNT(oi.OrderItemID) AS ItemCount,
    SUM(oi.Quantity * oi.UnitPrice) AS OrderTotal
FROM dbo.Orders o
JOIN dbo.Customers c ON o.CustomerID = c.CustomerID
JOIN dbo.OrderItems oi ON o.OrderID = oi.OrderID
GROUP BY o.OrderID, o.OrderDate, o.CustomerID, c.CustomerName;
GO
```

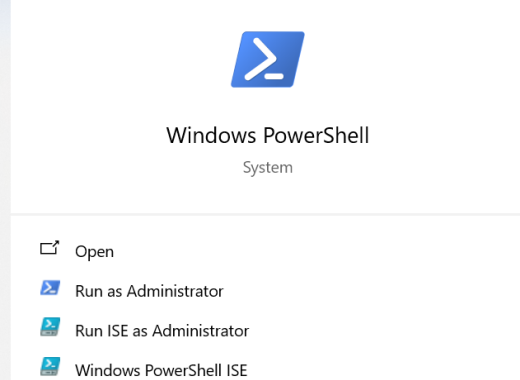
A.2 Stored procedure

```
CREATE OR ALTER PROCEDURE dbo.usp_GetOrderSummaryByDateRange
    @StartDate date, @EndDate date, @CustomerID int = NULL
AS
BEGIN
    SET NOCOUNT ON;
    IF @StartDate > @EndDate
        THROW 50001, 'StartDate must be less than or equal to EndDate.', 1;
    SELECT OrderID, OrderDate, CustomerID, CustomerName, ItemCount,
OrderTotal
    FROM dbo.vw_OrderSummary
    WHERE OrderDate >= @StartDate
      AND OrderDate <= @EndDate
      AND (@CustomerID IS NULL OR CustomerID = @CustomerID)
    ORDER BY OrderDate DESC;
END;
GO
```

End of Exercise 2. Explore, document, and build with GitHub Copilot.

Exercise 3 - Configuring AI in SQL Server

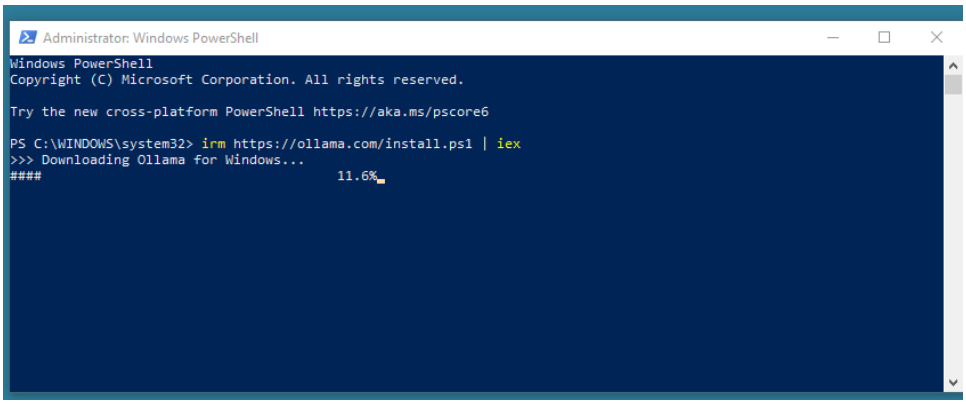
1. Open up PowerShell as administrator so that you can install Ollama, which is an open source AI model.



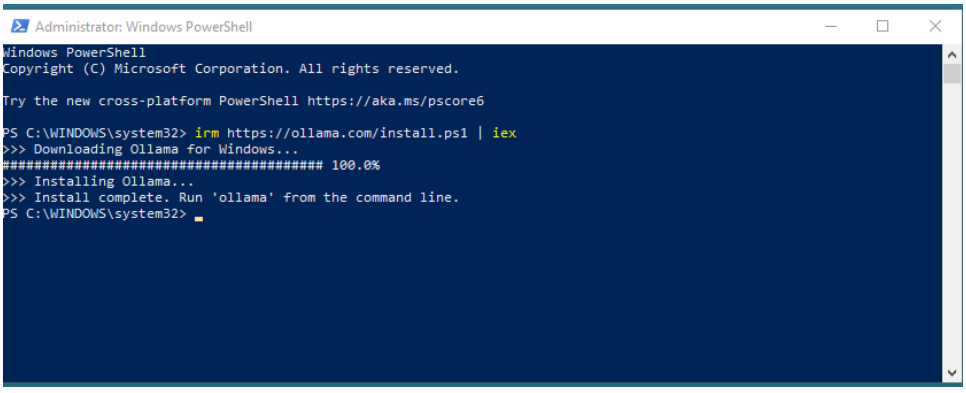
2. Enter the following command

```
irm https://ollama.com/install.ps1 | iex
```

Once you hit enter it will look like this



10 minutes later



3. Once the install is done, close and reopen PowerShell as administrator. This may take about 10 minutes.
4. In PowerShell, verify the installation by typing

```
ollama --version
```

5. Next, list the installed models

```
ollama list
```

This will be blank.

6. Verify that the Ollama service has started

```
Invoke-RestMethod `
    -Uri "http://localhost:11434/api/tags" `
    -Method Get
```

7. To use the sample model, you will need to pull it with this command

```
ollama pull all-minilm
```

Validate that the model has loaded with this command

```
ollama list
```

8. Test the newly installed model in PowerShell

```
$body = @{
    model = "all-minilm"
    input = "A lightweight mountain bicycle designed for rough trails."
} | ConvertTo-Json

$response = Invoke-RestMethod `
    -Uri "http://localhost:11434/api/embed" `
    -Method Post `
    -ContentType "application/json" `
    -Body $body

$response
```

9. Inspect the vectors

```
$response.embeddings[0]
```

10. This contains a number of vectors that you will need to use in SQL Server, so you need to know how many there are. Type this command in PowerShell.

```
$response.embeddings[0].Count
```

The number of vectors is the number you will use in SQL Server. The result you will see is 384.

11. Run the model in PowerShell to see how it can semantically encode data. This is the same thing you will do in SQL Server, but the data will be the contents of a table instead of the array you see here. Type this code into PowerShell.

```
$body = @{
    model = "all-minilm"
    input = @(
        "A mountain bicycle for rocky trails.",
        "A bike designed for off-road riding.",
        "A formal business suit.",
        "A replacement bicycle chain."
    )
} | ConvertTo-Json -Depth 4

$response = Invoke-RestMethod `
    -Uri "http://localhost:11434/api/embed" `
    -Method Post `
    -ContentType "application/json" `
    -Body $body

$response.embeddings.Count
```

Each string is going to have a vector, so with 4 lines of text, the value shown will be 4.

12. SQL Server refuses to make REST calls using just http. So, to get around this roadblock, for local hosting add a proxy so that it will make calls using https.

To fix this, download Caddy Server from https://caddyserver.com/

13. Create a file called caddyfile with the following contents

```
{
  auto_https disable_redirects
}

https://localhost:8443 {
  tls internal
  reverse_proxy 127.0.0.1:11434
}
```

14. Execute the file with this command

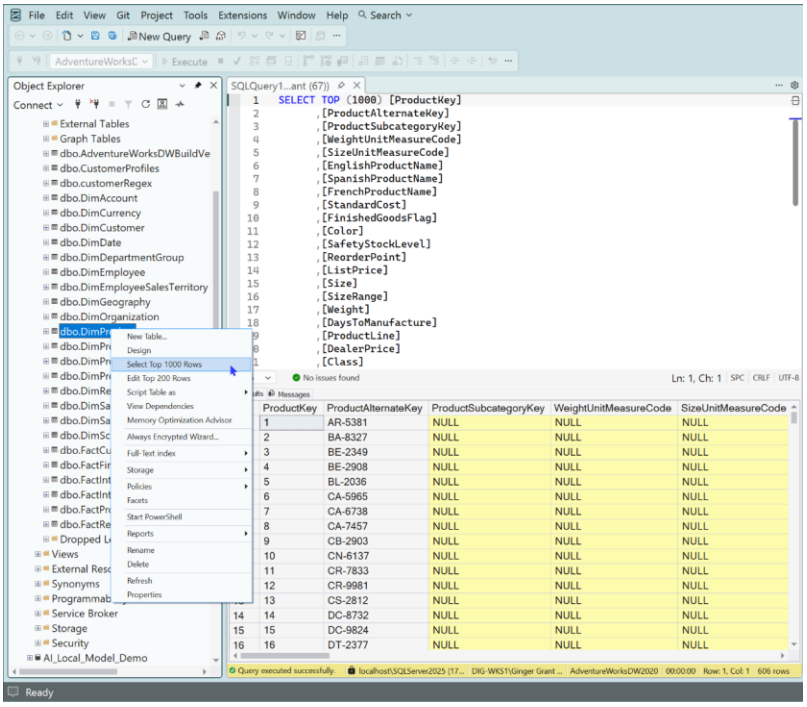
```
caddy_windows_amd64.exe run --config caddyfile
```

To see if it is running, in PowerShell run this command

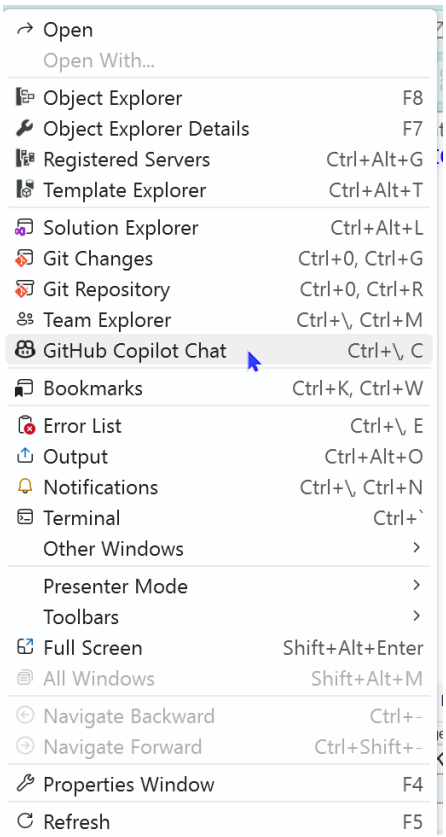
```
Get-Process | Where-Object {$_.ProcessName -like "*caddy*"}
```

15. Open up SSMS and the AdventureWorksDW2020 database you installed previously.

16. Right-click on the dbo.DimProduct table and select the option to Select Top 1000 Rows as shown in the image below.



17. Open GitHub Copilot chat by going to the View menu at the top of the screen and selecting it from the list as shown below.



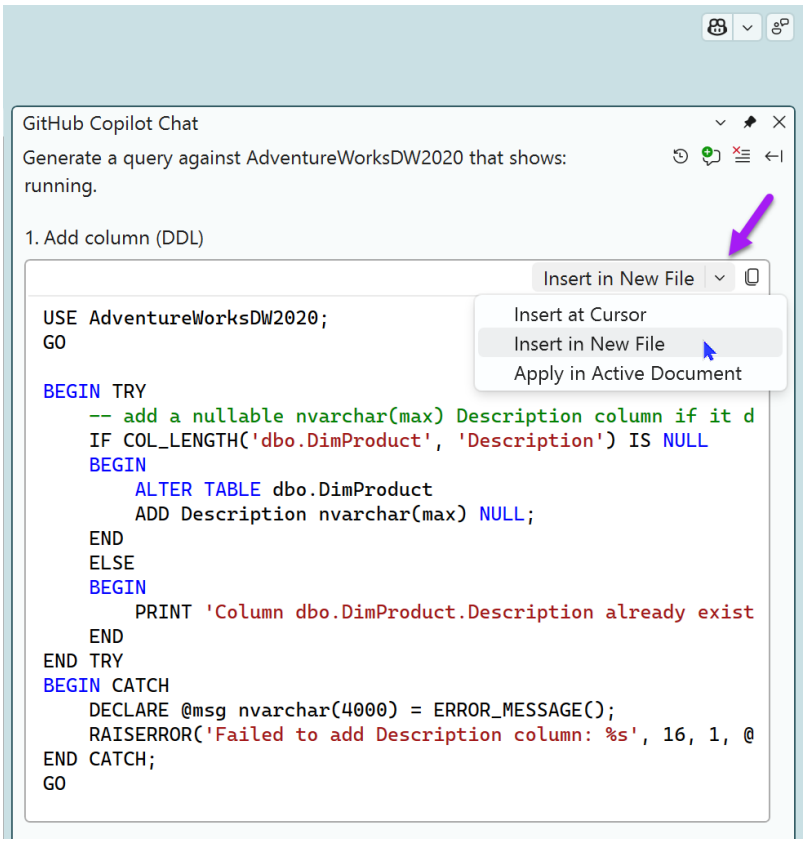
18. In the GitHub Copilot chat window, type the following prompt.

```
Create a script to update dbo.DimProduct to add a Description column
and fill the new column with a description based on the columns
EnglishProductCategoryName and EnglishProductSubcategoryName and
EnglishproductName in this query

SELECT a.[ProductKey], c.[EnglishProductCategoryName],
    A.EnglishproductName, b.[EnglishProductSubcategoryName]
FROM [AdventureWorksDW2020].[dbo].[DimProduct] a
Join [dbo].[DimProductSubcategory] b on a.ProductSubcategoryKey =
    b.ProductSubcategoryKey
Join dbo.[DimProductCategory] c on b.ProductCategoryKey =
    c.ProductCategoryKey

Use the matching productkey and generate a separate script to
populate it.
```

19. For each script that is generated, click the drop-down box in the script window as shown below and select Insert in New File as shown in the image here.



20. Run the script in SQL Server by selecting the execute button at the top of the query.
21. Complete these last two steps for both scripts.
22. Leave everything running for the next lab.

Exercise 4 - Using AI in SQL Server

1. Open up SSMS 22 and click on the New Query button to create a blank query.
2. In the empty query enter this query

```
ALTER DATABASE SCOPED CONFIGURATION SET PREVIEW_FEATURES = ON;
```

3. Create two EXTERNAL MODELS that reference two different models within Ollama to see which one performs better

```
CREATE EXTERNAL MODEL LocalOllama_Embeddings
WITH (
    LOCATION = 'https://localhost:8443/api/embed',
    API_FORMAT = 'Ollama',
    MODEL_TYPE = EMBEDDINGS,
    MODEL = 'nomic-embed-text'
);
GO

CREATE EXTERNAL MODEL LocalOllama_Embeddings_Large
WITH (
    LOCATION = 'https://localhost:8443/api/embed',
    API_FORMAT = 'Ollama',
    MODEL_TYPE = EMBEDDINGS,
    MODEL = 'mxbai-embed-large'
);
GO
```

4. Update the dbo.DimProduct table to include embeddings

```
UPDATE dbo.DimProduct
SET Embedding = AI_GENERATE_EMBEDDINGS(Description USE MODEL
LocalOllama_Embeddings)
WHERE Embedding IS NULL;
GO
```

5. Add a vector index

```
CREATE VECTOR INDEX IX_Product_Embedding
ON dbo.DimProduct (Embedding)
WITH (METRIC = 'cosine', TYPE = 'DiskANN');
GO
```

6. Search using the index

```
SELECT p.Product, s.distance
FROM VECTOR_SEARCH(
    TABLE = dbo.DimProduct AS p,
    COLUMN = Embedding_Large,
    SIMILAR_TO = @qvl2,
    METRIC = 'cosine', TOP_N = 5) AS s
ORDER BY s.distance;
```

Exercise 5 - Optimizing Semantic Search and Analytics in SQL Server 2025

Module 5: Performance and Query Optimization

Estimated Time: approximately 25 to 30 minutes.

Primary Tool: SQL Server Management Studio 22.

Platform: SQL Server 2025.

Lab Scenario: You are working on an AI-enabled product search application built on SQL Server 2025. The application stores product metadata, text descriptions, and vector embeddings. Users search for products using natural language, and the application returns semantically similar products. The initial implementation works, but performance becomes inconsistent as the product catalog grows. In this lab, you will use SSMS 22 to inspect the database design, compare query patterns, and apply optimization techniques for analytics and vector search workloads.

Learning Objectives

By completing this lab, you will be able to:

- Identify common bottlenecks in semantic search queries.
- Compare unfiltered and pre-filtered vector search patterns.
- Understand where approximate query processing can improve analytical workloads.
- Evaluate when columnstore indexes are useful for analytics.
- Apply caching concepts for AI-generated embeddings or repeated semantic search requests.
- Use SSMS 22 to execute, inspect, and compare query behavior.

Prerequisites

Before starting this lab, students should have:

- SQL Server 2025 environment available.
- SSMS 22 installed.
- Workshop database restored or created.
- Sample product catalog table available.
- Sample embeddings already generated, or a script available that populates embedding data.
- Basic familiarity with SELECT, WHERE, ORDER BY, indexes, and execution plans.

Suggested Database Objects

The lab can use the following database objects:

dbo.Products: ProductID, ProductName, Description, Category, Price, StockQuantity, LastUpdated.

dbo.ProductEmbeddings: ProductID, EmbeddingVector, EmbeddingModel, EmbeddingCreatedDate, SourceHash.

dbo.SemanticSearchCache: CacheID, SearchText, SearchTextHash, QueryEmbedding, ResultJson, ModelName, CreatedDate, ExpiresDate.

dbo.SearchLog: used for logging search activity and query metadata.

dbo.ProductAnalytics: used for analytical queries across product and search data.

Part 1: Open the Environment in SSMS 22

Goal: Connect to the SQL Server 2025 instance and review the lab database.

Steps

1. Open SQL Server Management Studio 22.
2. Connect to the SQL Server 2025 instance provided by the instructor.
3. Open Object Explorer.
4. Expand the workshop database.
5. Review the tables related to products, embeddings, search logs, and cache data.
6. Open a new query window connected to the workshop database.

Discussion Questions

- Which tables contain transactional product data?
- Which tables contain AI or vector-related data?
- Which columns are likely to be used for filtering?
- Which columns are likely to be expensive to compute or compare?

Part 2: Review the Baseline Semantic Search Query

Goal: Understand the baseline search query before optimization.

Steps

1. Open the provided baseline semantic search query.
2. Review how the query compares the user query vector to product embedding vectors.
3. Identify whether the query filters by category, price, stock, or other metadata.
4. Execute the query in SSMS 22.
5. Review the result set.
6. Enable the actual execution plan if instructed by the instructor.
7. Re-run the query and observe the plan shape.

Observations to Record

- Does the query compare against all available vectors?
- Does the query apply metadata filters before or after vector distance calculation?
- Are there obvious scans?
- Are there opportunities to reduce the number of rows before vector comparison?

Part 3: Add Metadata Pre-Filtering

Goal: Improve semantic search performance by reducing the candidate set before vector similarity scoring.

Steps

1. Modify the baseline query to filter products before vector comparison.
2. Add filters such as category, price range, and stock availability.

3. Execute the filtered query.
4. Compare the result set with the baseline query.
5. Compare query behavior and execution characteristics.

Example Pattern

Use a query structure that follows this logic:

```
-- Step 1: Identify candidate products using metadata filters
WITH CandidateProducts AS (
    SELECT ProductID, ProductName, Category, Price, StockQuantity
    FROM dbo.Products
    WHERE Category = @Category
      AND Price BETWEEN @MinPrice AND @MaxPrice
      AND StockQuantity > 0
)
-- Step 2: Join candidates to their embeddings
-- Step 3: Calculate vector distance only for the filtered set
-- Step 4: Sort by similarity
-- Step 5: Return the top matches
SELECT TOP 10
    cp.ProductID,
    cp.ProductName,
    VECTOR_DISTANCE('cosine', pe.EmbeddingVector, @QueryVector) AS Similarity
FROM CandidateProducts cp
JOIN dbo.ProductEmbeddings pe ON pe.ProductID = cp.ProductID
ORDER BY Similarity ASC;
```

Observations to Record

- How many rows are considered before vector comparison?
- Did filtering change the quality of the results?
- Did filtering make the query easier to reason about?
- Which filters are most useful for your scenario?

Part 4: Evaluate Indexing Strategy

Goal: Understand which indexes support hybrid semantic search workloads.

Steps

1. Review existing indexes on the product and embedding tables.
2. Identify indexes that support common metadata filters.
3. Review whether indexes exist on Category, Price, StockQuantity, ProductID, and LastUpdated.
4. Discuss where a columnstore index may help analytical queries.
5. Discuss where a vector index or approximate nearest-neighbor strategy may help vector search.

Index Strategy Reference

Transactional (point lookups, row inserts/updates): traditional B-tree rowstore indexes on primary key and foreign keys.

Metadata filtering (Category, Price, Stock): non-clustered indexes on filter columns; consider composite indexes for multi-column filters.

Analytical (aggregations, grouping, large scans): columnstore index, which enables batch-mode processing and high compression.

Semantic / vector similarity search: vector index or approximate nearest-neighbor (ANN) strategy to avoid full-scan distance calculations.

Hybrid (filter + vector): pre-filter via rowstore indexes, then apply vector distance on the reduced candidate set.

Discussion Questions

- Which queries are transactional?
- Which queries are analytical?
- Which queries are semantic or vector-based?
- Which indexes support filtering?
- Which indexes support aggregation?
- Which indexes support similarity search?

Part 5: Explore Columnstore for Analytics

Goal: Understand how columnstore indexes support analytical queries.

Steps

1. Open the provided analytics query.
2. Review aggregations by category, price band, stock level, or search activity.
3. Execute the query against the current table design.
4. Review whether the query scans many rows.
5. Discuss whether columnstore indexing is appropriate for this workload.
6. If the instructor provides a columnstore version of the table or index script, compare the analytical query behavior.

Columnstore vs. Rowstore: Key Differences

Storage layout: columnstore data is stored by column, ideal for reading a few columns across many rows.

Compression: high compression ratio, reduces I/O significantly for wide tables.

Processing mode: batch-mode execution, processes thousands of rows per operation.

Best for: aggregations (SUM, AVG, COUNT), GROUP BY, large analytical scans.

Not suited for: single-row lookups, frequent inserts and updates on the same rows.

Observations to Record

- Is the query scanning many rows?
- Does the query aggregate over large volumes of data?
- Is the query more analytical than transactional?
- Would columnstore compression and batch-mode processing be useful here?

Part 6: Add a Caching Pattern

Goal: Reduce repeated AI and vector processing by caching repeated semantic search requests.

Steps

1. Review the semantic search cache table (dbo.SemanticSearchCache).
2. Identify the cache key, such as a normalized search phrase or hash.
3. Review how cached results could be stored as JSON.
4. Modify or review the provided stored procedure pattern:

```
-- Stored procedure caching pattern
CREATE OR ALTER PROCEDURE dbo.SearchProducts
    @SearchText NVARCHAR(500),
    @QueryVector VECTOR(1536)
AS
BEGIN
    -- Step 1: Check cache first
    DECLARE @Hash VARBINARY(32) = HASHBYTES('SHA2_256',
LOWER(LTRIM(RTRIM(@SearchText))));

    SELECT ResultJson
    FROM dbo.SemanticSearchCache
    WHERE SearchTextHash = @Hash
      AND ExpiresDate > GETUTCDATE();

    IF @@ROWCOUNT > 0 RETURN;  -- Cache hit, return early

    -- Step 2: Execute semantic search
    -- ... (pre-filtered vector query here)

    -- Step 3: Store result in cache
    INSERT INTO dbo.SemanticSearchCache
        (SearchText, SearchTextHash, QueryEmbedding, ResultJson, ModelName,
CreatedDate, ExpiresDate)
    VALUES (@SearchText, @Hash, @QueryVector, @ResultJson, @ModelName,
        GETUTCDATE(), DATEADD(HOUR, 24, GETUTCDATE()));
END;
```

5. Execute the procedure or script provided by the instructor.
6. Run the same search more than once and discuss expected behavior.

Discussion Questions

- What should be cached: embeddings, search results, or model responses?
- How long should cache entries remain valid?
- How should source data changes invalidate cache entries?
- Should model version be part of the cache key?
- What are the risks of returning stale semantic search results?

Part 7: Wrap-Up Review

Student Deliverables

Students should be able to explain:

- The difference between baseline and pre-filtered semantic search.
- Why metadata filtering improves vector search performance.
- When approximate query processing is appropriate.

- When columnstore indexing helps analytics.
- Why caching is important for AI-enabled database workloads.
- How SSMS 22 can be used to inspect and compare query behavior.

Instructor Notes

Key Teaching Point: Emphasize that performance optimization is not a single feature. It is a design discipline involving data modeling, indexing, query patterns, caching, and workload awareness.

For AI-enabled SQL Server applications, the most important performance question is often: "How much work can we avoid doing repeatedly?"

Lab Summary by Part

Part 1, Environment Setup: familiarity with the database schema and tooling.

Part 2, Baseline Query: understanding unoptimized vector search patterns.

Part 3, Pre-Filtering: reducing candidate rows before expensive vector distance calculation.

Part 4, Indexing Strategy: choosing the right index for each query workload type.

Part 5, Columnstore Analytics: batch-mode processing and compression for analytical queries.

Part 6, Caching Pattern: avoiding repeated AI and vector processing through result reuse.

Part 7, Wrap-Up: student synthesis and delivery of key concepts.

End of Exercise 5.